```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.LinkedHashMap;

public class PassOne {
   int lc=0;
   int libtab_ptr=0,pooltab_ptr=0;
   int symIndex=0,litIndex=0;
   LinkedHashMap<String, TableRow> SYMTAB;
   ArrayList<TableRow> LITTAB;
   ArrayList<Integer> POOLTAB;
   private BufferedReader br;

   public PassOne()
   {
      SYMTAB =new LinkedHashMap<>();
      LITTAB=new ArrayList<>();
      POOLTAB=new ArrayList<>();
      lc=0;
      POOLTAB.add(0);
   }
   public static void main(String[] args) {
      PassOne one=new PassOne();
      try
      {
         one.parseFile();
```

```
    }
    catch (Exception e) {
       System.out.println("Error: "+e);  //
    }
}
public void parseFile() throws Exception
{ String prev="";
    String line,code;
    br = new BufferedReader(new FileReader("sample.asm"));
    BufferedWriter bw=new BufferedWriter(new FileWriter("IC.txt"));
    INSTtable lookup=new INSTtable();
    while((line=br.readLine())!=null)
    {

       String parts[]=line.split("\\s+");
       if(!parts[0].isEmpty()) //processing of label
       {
          if(SYMTAB.containsKey(parts[0]))
             SYMTAB.put(parts[0], new TableRow(parts[0], lc, SYMTAB.get(parts[0]).getIndex()));
          else
             SYMTAB.put(parts[0],new TableRow(parts[0], lc, ++symIndex));
       }

       if(parts[1].equals("LTORG"))
       {
          int ptr=POOLTAB.get(pooltab_ptr);
          for(int j=ptr;j<libtab_ptr;j++)
          {
             lc++;
             LITTAB.set(j, new TableRow(LITTAB.get(j).getSymbol(),lc));
             code="(DL,01)\t(C,"+LITTAB.get(j).symbol+")";
```

```
        bw.write(code+"\n");
    }
    pooltab_ptr++;
    POOLTAB.add(libtab_ptr);
}
if(parts[1].equals("START"))
{
    lc=expr(parts[2]);
    code="(AD,01)\t(C,"+lc+")";
    bw.write(code+"\n");
    prev="START";
}
else if(parts[1].equals("ORIGIN"))
{
    lc=expr(parts[2]);
    String splits[]=parts[2].split("\\+"); //Same for - SYMBOL //Add code
    code="(AD,03)\t(S,"+SYMTAB.get(splits[0]).getIndex()+")"+Integer.parseInt(splits[1]);
    bw.write(code+"\n");
}

//Now for EQU
if(parts[1].equals("EQU"))
{
    int loc=expr(parts[2]);
    //below If conditions are optional as no IC is generated for them
    if(parts[2].contains("+"))
    {
        String splits[]=parts[2].split("\\+");
        code="(AD,04)\t(S,"+SYMTAB.get(splits[0]).getIndex()+")"+Integer.parseInt(splits[1]);

    }
```

```
    else if(parts[2].contains("-"))
    {
       String splits[]=parts[2].split("\\-");
       code="(AD,04)\t(S,"+SYMTAB.get(splits[0]).getIndex()+")-"+Integer.parseInt(splits[1]);
    }
    else
    {
       code="(AD,04)\t(C,"+Integer.parseInt(parts[2]+")");
    }
    bw.write(code+"\n");
    if(SYMTAB.containsKey(parts[0]))
       SYMTAB.put(parts[0], new TableRow(parts[0],loc,SYMTAB.get(parts[0]).getIndex())) ;
    else
       SYMTAB.put(parts[0], new TableRow(parts[0],loc,++symIndex));
}

if(parts[1].equals("DC"))
{
    lc++;
    int constant=Integer.parseInt(parts[2].replace("'",""));
    code="(DL,01)\t(C,"+constant+")";
    bw.write(code+"\n");
}
else if(parts[1].equals("DS"))
{

    int size=Integer.parseInt(parts[2].replace("'", ""));

    code="(DL,02)\t(C,"+size+")";
    bw.write(code+"\n");
    /*if(prev.equals("START"))
```

```
            {
               lc=lc+size-1;//System.out.println("here");


            }
            else
*/             lc=lc+size;
            prev="";
         }
         if(lookup.getType(parts[1]).equals("IS"))
         {
            code="(IS,0"+lookup.getCode(parts[1])+")\t";
            int j=2;
            String code2="";
            while(j<parts.length)
            {
               parts[j]=parts[j].replace(",", "");
               if(lookup.getType(parts[j]).equals("RG"))
               {
                  code2+=lookup.getCode(parts[j])+"\t";
               }
               else
               {
                  if(parts[j].contains("="))
                  {
                     parts[j]=parts[j].replace("=", "").replace("'", "");
                     LITTAB.add(new TableRow(parts[j], -1,++litIndex));
                     libtab_ptr++;
                     code2+="(L,"+(litIndex)+")";
                  }
                  else if(SYMTAB.containsKey(parts[j]))
                  {
```

```
            int ind=SYMTAB.get(parts[j]).getIndex();
            code2+= "(S,0"+ind+")";
          }
          else
          {
            SYMTAB.put(parts[j], new TableRow(parts[j],-1,++symIndex));
            int ind=SYMTAB.get(parts[j]).getIndex();
            code2+= "(S,0"+ind+")";
          }
        }
        j++;
    }
    lc++;
    code=code+code2;
    bw.write(code+"\n");
}

if(parts[1].equals("END"))
{
    int ptr=POOLTAB.get(pooltab_ptr);
    for(int j=ptr;j<libtab_ptr;j++)
    {
        lc++;
        LITTAB.set(j, new TableRow(LITTAB.get(j).getSymbol(),lc));
        code="(DL,01)\t(C,"+LITTAB.get(j).symbol+")";
        bw.write(code+"\n");
    }
    pooltab_ptr++;
    POOLTAB.add(libtab_ptr);
    code="(AD,02)";
    bw.write(code+"\n");
```

```
        }


    }
    bw.close();
    printSYMTAB();
    //Printing Literal table
    PrintLITTAB();
    printPOOLTAB();
}
void PrintLITTAB() throws IOException
{
    BufferedWriter bw=new BufferedWriter(new FileWriter("LITTAB.txt"));
    System.out.println("\nLiteral Table\n");
    //Processing LITTAB
    for(int i=0;i<LITTAB.size();i++)
    {
        TableRow row=LITTAB.get(i);
        System.out.println(i+"\t"+row.getSymbol()+"\t"+row.getAddess());
        bw.write((i+1)+"\t"+row.getSymbol()+"\t"+row.getAddess()+"\n");
    }
    bw.close();
}
void printPOOLTAB() throws IOException
{
    BufferedWriter bw=new BufferedWriter(new FileWriter("POOLTAB.txt"));
    System.out.println("\nPOOLTAB");
    System.out.println("Index\t#first");
    for (int i = 0; i < POOLTAB.size(); i++) {
        System.out.println(i+"\t"+POOLTAB.get(i));
        bw.write((i+1)+"\t"+POOLTAB.get(i)+"\n");
    }
```

```
    bw.close();
}
void printSYMTAB() throws IOException
{
    BufferedWriter bw=new BufferedWriter(new FileWriter("SYMTAB.txt"));
    //Printing Symbol Table
    java.util.Iterator<String> iterator = SYMTAB.keySet().iterator();
    System.out.println("SYMBOL TABLE");
    while (iterator.hasNext()) {
        String key = iterator.next().toString();
        TableRow value = SYMTAB.get(key);

        System.out.println(value.getIndex()+"\t" + value.getSymbol()+"\t"+value.getAddess());
        bw.write(value.getIndex()+"\t" + value.getSymbol()+"\t"+value.getAddess()+"\n");
    }
    bw.close();
}
public int expr(String str)
{
    int temp=0;
    if(str.contains("+"))
    {
        String splits[]=str.split("\\+");
        temp=SYMTAB.get(splits[0]).getAddess()+Integer.parseInt(splits[1]);
    }
    else if(str.contains("-"))
    {
        String splits[]=str.split("\\-");
        temp=SYMTAB.get(splits[0]).getAddess()-(Integer.parseInt(splits[1]));
    }
    else
```

```
        {
            temp=Integer.parseInt(str);
        }
        return temp;
    }
}


import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.HashMap;

public class Pass2 {
    private static final String INTERMEDIATE_CODE_FILE = "/home/student/IC.txt";
    private static final String SYM_TAB_FILE = "/home/student/SYMTAB.txt";
    private static final String LITERALS_FILE = "/home/student/LITTAB.txt";
    private static final String OUTPUT_FILE = "/home/student/Pass2.txt";

    public static void main(String[] args) {
        HashMap<Integer, String> symSymbol = new HashMap<>();
        HashMap<Integer, String> litAddr = new HashMap<>();

        try (
            BufferedReader b1 = new BufferedReader(new FileReader(INTERMEDIATE_CODE_FILE));
            BufferedReader b2 = new BufferedReader(new FileReader(SYM_TAB_FILE));
            BufferedReader b3 = new BufferedReader(new FileReader(LITERALS_FILE));
            FileWriter f1 = new FileWriter(OUTPUT_FILE)
        ) {
            String line;
```

```
// Read symbol table: format assumed [index] [address]
while ((line = b2.readLine()) != null) {
   String[] words = line.trim().split("\\s+");
   if (words.length >= 2) {
      int index = Integer.parseInt(words[0]);
      symSymbol.put(index, words[1].trim());
   }
}

// Read literals: format assumed [literal] [address]
int litIndex = 1;
while ((line = b3.readLine()) != null) {
   String[] words = line.trim().split("\\s+");
   if (words.length >= 2) {
      litAddr.put(litIndex++, words[1].trim());
   }
}

// Process intermediate code
while ((line = b1.readLine()) != null) {
   line = line.trim();
   if (line.isEmpty()) continue;

   String[] parts = line.split("\\s+");
   StringBuilder output = new StringBuilder("+ ");

   for (String part : parts) {
      part = part.trim();

      if (part.startsWith("(IS,")) {
```

```
                String opcode = part.substring(4, part.length() - 1);
                output.append(opcode).append(" ");
            } else if (part.matches("\\d+")) {
                output.append(part).append(" ");
            } else if (part.startsWith("(R,")) {
                String reg = part.substring(3, part.length() - 1);
                output.append(reg).append(" ");
            } else if (part.startsWith("(S,")) {
                int index = Integer.parseInt(part.substring(3, part.length() - 1));
                output.append(symSymbol.getOrDefault(index, "000")).append(" ");
            } else if (part.startsWith("(L,")) {
                int index = Integer.parseInt(part.substring(3, part.length() - 1));
                output.append(litAddr.getOrDefault(index, "000")).append(" ");
            } else if (part.startsWith("(C,")) {
                String constant = part.substring(3, part.length() - 1);
                output.append(constant).append(" ");
            } else if (part.contains("+")) {
            // Handles expressions like (S,1)+1
            String[] expr = part.split("\\+");
            String symbolPart = expr[0].trim();
             String offsetPart = expr[1].trim();

   if (symbolPart.startsWith("(S,") && symbolPart.endsWith(")")) {
      int index = Integer.parseInt(symbolPart.substring(3, symbolPart.length() - 1));
      int base = Integer.parseInt(symSymbol.getOrDefault(index, "0"));
      int offset = Integer.parseInt(offsetPart);
      output.append(base + offset).append(" ");
   }
}
        }
```

```
            f1.write(output.toString().trim() + "\n");
        }

    } catch (IOException e) {
        System.err.println("Error occurred while processing files: " + e.getMessage());
    }
  }
}
```